

Security-Relevant Evolution of a GitSkills

SWE 380 / CSC 580 Group 8 Project Report

Jim Prantzos

Kyle Cantrell

Vaisnavii Mohanraj

Christian Heiney

Fall 2026

Abstract

[TODO: In approximately 150-250 words, summarize the research problem, research question, dataset, method, principal results, and implications. Complete this section after the analysis is finished.]

Introduction

[TODO: Introduce AI-agent skills, GitSkills, and the broader software-engineering context of the project.]

Background and Context

GitSkills artifacts can contain instructions that direct AI agents to execute commands, access or modify files, communicate with external services, invoke scripts, or handle credentials. These behaviors are often legitimate and necessary, but they are security-relevant because carrying them out could affect a system, its data, or external resources.

Skills may also be modified over time or reused across repositories. During that process, security-relevant instructions may be added, removed, or altered. A modified skill could introduce command execution, file-system access, network access, script use, credential handling, or another consequential behavior that was not present in an earlier version or related source artifact.

Motivation

Studying these changes can provide evidence about how security-relevant behavior evolves within GitSkills and whether reuse or modification introduces new risk signals. A reproducible analysis can also help developers and reviewers identify changes that may warrant closer security review.

This topic also has personal relevance to the team because two members are pursuing cybersecurity concentrations. It provides an opportunity to apply security-focused coursework to an emerging software-engineering problem while developing practical experience in threat identification, static analysis, and responsible interpretation of security evidence.

Expected Contribution

This project is expected to produce:

1. A documented taxonomy of security-relevant instruction patterns, including command execution, file-system operations, external network access, script use, and credential-related instructions.
2. A reproducible, rule-based static-analysis pipeline for detecting those patterns in GitSkills artifacts without executing untrusted instructions or scripts.
3. A reproducible method for identifying and comparing related skill artifacts across versions or reuse relationships.
4. An analysis showing whether security-relevant instruction patterns are introduced, removed, retained, or modified between earlier or source artifacts and later or related artifacts.
5. Empirical measurements describing the prevalence of each defined pattern and the frequency with which new patterns appear during observed skill evolution or reuse.
6. A validation and reporting approach that documents detection limitations, relationship uncertainty, ambiguous cases, and false positives while distinguishing observable risk signals from confirmed vulnerabilities or malicious behavior.

Together, these contributions will provide a repeatable method for examining both the presence and evolution of security-relevant instructions in GitSkills artifacts.

Report Organization

[TODO: Briefly describe how the remainder of the report is organized.]

Research Question and Expectations

Selected Topic

This project investigates Question 4, **Skill security and supply-chain risk**, from the MSR 2027 Mining Challenge-inspired project outline.

Research Question

Do modified or reused skills introduce command execution, file-system access, network access, or other risky behavior that was absent from an earlier version or source artifact?

Expectations

[TODO: Describe the patterns the team expects to observe. Clearly distinguish expectations from findings.]

Operational Definitions

Base Artifact

[TODO: Define a base artifact and explain how it is identified.]

Derived Artifact

[TODO: Define a derived artifact and explain how it is related to the base artifact.]

Security-Relevant Instruction Pattern

[TODO: Define a security-relevant instruction pattern without treating it as proof of maliciousness or exploitability.]

Introduced Risk Signal

[TODO: Define what must be detected in the derived artifact and absent from the base artifact for a signal to count as introduced.]

Modified or Reused Skill

[TODO: Define the relationships included in the study and the evidence used to establish them.]

Dataset and Data Preparation

This section describes the dataset source, study scope, data schema, sampling strategy, eligibility filters, preprocessing steps, and safety controls used to construct the analysis-ready data. The detailed algorithms and implementation pipeline are described in the Method and Implementation section.

Dataset Source and Acquisition

The source dataset for this study is `agent_skills_release.db`, the GitSkills database provided for the MSR 2027 Mining Challenge. The source population consists of all GitSkills artifact records contained in the database.

[TODO: Add the approved dataset access instructions, release or snapshot identifier, file integrity information, and the date on which the database was obtained. Explain that the full database is not committed to the project repository and identify the expected local storage location.]

Dataset Scope

The study distinguishes three populations:

1. **Source population:** All GitSkills artifact records in `agent_skills_release.db`.
2. **Analyzable artifact population:** Records from the source population that contain stored skill content that the static analyzer can process.
3. **Eligible comparison population:** Ordered base-derived comparisons that can be extracted from the database and satisfy the eligibility criteria defined below.

Artifacts without analyzable content or without an identifiable, ordered relationship remain part of the source population, but they are not part of the eligible comparison population. The pipeline will report the number of records and candidate relationships excluded at each stage and the reasons for exclusion.

[TODO: Insert verified counts for the source population, analyzable artifacts, unique content SHAs, candidate relationships, eligible comparisons, repositories, and other scope measures produced by the final pipeline.]

Relevant Schema

The analysis uses data at four related levels: artifact, relationship, scanner profile, and ordered comparison. Exact source-table and column names will be verified against the implemented extraction pipeline and documented in `DATA_DICTIONARY.md`.

Artifact Data

For both base and derived artifacts, the prepared data retains the available fields needed for identification, analysis, and traceability:

- artifact identifier;
- repository identifier;
- artifact path;
- declared skill name, when available;
- content SHA;
- stored content or a safe reference to the analyzed content; and
- available timestamp, commit, or other ordering metadata used by the relationship method.

Relationship Data

Each candidate or eligible relationship records:

- comparison identifier;
- base artifact identifier;
- derived artifact identifier;
- relationship type;
- same-repository or cross-repository status;
- relationship-detection method;
- evidence supporting the relationship;
- evidence supporting base-derived ordering;
- relationship confidence or review status, if used by the implemented method; and
- eligibility or exclusion status and reason.

Scanner-Profile Data

For each analyzed artifact, the prepared scanner profile contains:

- matched rule identifiers;
- matched security-relevant categories;
- match counts by rule and category;
- total matched-rule and matched-category counts;
- match locations or other evidence required for validation and safe reporting; and
- scanner and rule-catalog versions.

Comparison Data

For each eligible ordered comparison, the prepared analysis table contains:

- introduced, removed, and retained rule identifiers;
- introduced, removed, and retained categories;
- counts of introduced rules and categories;
- indicators for at least one introduced rule and at least one introduced category; and
- the overall comparison result under the documented threat model.

Rule-level and category-level fields are both retained. A derived artifact may introduce a new rule within a category that was already present in the base artifact; that change is visible at the rule level even when the set of categories does not change.

Unit of Analysis

The primary unit of analysis is one ordered comparison between two related, content-bearing skill artifacts:

- **Base artifact:** The earliest identified version or source content in the relationship.
- **Derived artifact:** The later, reused, or modified content being compared with the base.

The ordering is represented as:

base artifact -> derived artifact

One unique content SHA is the secondary processing unit. The pipeline may scan identical content once and reuse its profile, but SHA-based deduplication is only a computational optimization. Artifact occurrences, repository occurrences, and comparison relationships remain available after cached profiles are joined back to the complete extracted data.

Inclusion and Exclusion Criteria

An ordered comparison is eligible when all of the following conditions are satisfied:

1. Both artifacts contain stored content that the scanner can analyze.
2. The extraction pipeline identifies a relationship using the project's documented relationship method.
3. Available evidence supports the base-derived ordering.
4. Both artifacts can be processed with the same scanner version and rule-catalog version.

A candidate comparison is excluded when one or more of these conditions are not satisfied. The pipeline retains the exclusion status and reason so that filtering is auditable. Uncertain or invalid relationships may be reviewed and excluded according to the documented relationship method.

[TODO: Add any implemented content-format, parsing, duplication, missing-value, or relationship-confidence filters and the exact exclusion codes used by the pipeline.]

Data Preparation

The preparation process converts the source database into analysis-ready artifact and comparison data. It will:

1. extract artifact content and traceability metadata from the relevant source tables;
2. identify candidate relationships and retain the evidence used to establish each relationship;
3. determine base-derived ordering from the available evidence;
4. apply the documented inclusion and exclusion criteria;
5. normalize content only as required by the scanner while preserving the original content or a safe reference to it;
6. identify unique content SHAs for cached scanning without discarding artifact occurrences;
7. attach scanner profiles to artifact records; and
8. join base and derived profiles into the ordered comparison table.

[TODO: Document the implemented text decoding, line-ending normalization, parsing, missing-content handling, and any other cleaning transformations. Distinguish transformations used for matching from the original content retained for traceability.]

Population Analysis and Sampling

Automated Census

The automated analysis will attempt a census of the eligible comparison population rather than select a statistical sample. Every eligible base-derived comparison extracted from `agent_skills_release.db` will be processed by the comparison pipeline.

Smaller reproducible datasets may be used while developing and debugging the pipeline. These development datasets are test inputs, not the research sample, and will not replace the final population-level analysis.

Manual Validation Sample

The team will manually review a stratified sample of at least 100 eligible base-derived comparisons from the automated results:

- at least 50 comparisons for which the scanner reports one or more newly introduced rules or categories; and
- at least 50 comparisons for which the scanner reports no newly introduced rules or categories.

Within those two groups, the sample will represent the relationship types and scanner categories present in the results. Selection will use a recorded random seed after the strata are defined. If a stratum contains fewer cases than its allocation, all cases in that stratum will be reviewed and the remaining allocation will be redistributed using a documented procedure.

The sample may be expanded if early review identifies unstable rules, poorly represented relationship types, or categories that require additional evidence. The manual-review procedure, labels, metrics, and error analysis belong in the Evaluation and Validation section.

For each sampled comparison, reviewers will record:

- whether the relationship and base-derived ordering are supported by the retained evidence;
- whether each reported match corresponds to the text identified by the scanner;
- whether the assigned categories are appropriate;
- whether a reported introduction was absent from the base and present in the derived artifact;
- whether an unflagged comparison contains a relevant introduction missed by the current rules; and
- whether the case is a true positive, false positive, false negative, true negative, or ambiguous under the documented threat model.

These records will support confirmed and rejected relationship counts, ordering disagreements, classification counts, precision and any other justified validation metrics, category- or relationship-specific breakdowns where sample size permits, and representative failure modes. The Evaluation and Validation section will define the final calculation and interpretation of those measures.

Data Ethics and Safety

All artifact content is treated as untrusted data. The project performs static analysis only and will not execute scripts, commands, or instructions contained in the dataset.

The pipeline and reporting process will:

- avoid executing or importing untrusted artifact code;
- avoid exposing credentials, tokens, or other sensitive strings that may appear in artifact content;
- retain only the match locations or excerpts needed for validation and safe reporting;
- distinguish detected risk signals from proof of malicious intent, vulnerability, exploitability, or real-world harm; and
- report aggregate findings and carefully selected examples using the project’s responsible reporting framework.

[TODO: Document the implemented redaction, access-control, retention, and safe-example-selection procedures.]

Method and Implementation

Method Overview

The study measures whether modified or reused GitSkills introduce security-relevant behavior that was absent from an earlier version or source artifact. For each eligible ordered comparison, the method scans the base and derived artifacts with the same versioned rule catalog, constructs a rule and category profile for each artifact, and measures which signals were introduced, removed, or retained in the derived artifact.

The threat model and rule catalog are still under development. The method therefore depends on stable rule identifiers, higher-level categories, and versioned configurations rather than on a fixed preliminary list of checks. Rules and categories may be refined without changing the overall comparison design, but a reported pipeline run must use one documented scanner and rule-catalog version.

Analysis Algorithm

Artifact Profiling

Let $R(a)$ be the set of rule identifiers detected in artifact (a), and let $C(a)$ be the set of security-relevant categories represented by those rules. For each artifact, the static analyzer records the matched rules, categories, counts, and match evidence required for validation.

Base-Derived Comparison

For an eligible ordered pair with base artifact (b) and derived artifact (d), the comparison stage computes:

$$R_{introduced} = R(d) \setminus R(b)$$

$$R_{removed} = R(b) \setminus R(d)$$

$$R_{retained} = R(b) \cap R(d)$$

The same set operations are applied to category profiles:

$$C_{introduced} = C(d) \setminus C(b), \quad C_{removed} = C(b) \setminus C(d), \quad C_{retained} = C(b) \cap C(d)$$

A comparison has a rule-level introduction when $|R_{introduced}| > 0$, and it has a category-level introduction when $|C_{introduced}| > 0$. Both measures are retained because a new rule may appear within a category already present in the base artifact.

Outcome Computation

The primary outcome is the proportion of eligible comparisons in which the derived artifact contains at least one detected security-relevant signal that was absent from the base:

$$\text{Rule introduction rate} = \frac{\text{eligible comparisons with at least one introduced rule}}{\text{eligible comparisons analyzed}}$$

$$\text{Category introduction rate} = \frac{\text{eligible comparisons with at least one introduced category}}{\text{eligible comparisons analyzed}}$$

Secondary outputs may include:

- the count and percentage of comparisons introducing each rule or category;
- distributions of introduced-rule and introduced-category counts;
- counts and percentages of comparisons with removed signals;
- counts and percentages with retained signals but no introduced signals;
- counts and percentages with no detected profile change;
- introduction rates by relationship type;
- introduction rates for same-repository and cross-repository comparisons; and
- the numbers of distinct artifacts, content SHAs, and repositories represented.

Every reported denominator will identify whether it represents comparison occurrences, unique artifact relationships, or unique content-SHA pairs.

Threat Model and Rule Catalog

The static analyzer is intended to identify security-relevant instructions or behaviors such as executable commands, URLs, scripts, file operations, credential-related instructions, and other categories adopted by the final threat model. A rule match is a reviewable signal, not a conclusion that an artifact is malicious or unsafe.

Each rule in the catalog should define:

- a stable rule identifier;
- rule name and description;
- higher-level security-relevant category;
- detection logic or pattern reference;
- severity or review priority, if adopted by the threat model;
- known false-positive conditions;
- positive and negative examples; and
- rule-catalog version.

[TODO: Insert the finalized threat-model scope, category definitions, rationale for inclusion, and the path to the version-controlled rule catalog.]

Static-Analysis Algorithm

The scanner processes artifact content as text and produces rule-level and category-level profiles without executing the content. Each match retains sufficient location or excerpt evidence for validation and safe reporting. The base and derived artifacts in one comparison are always scanned with the same scanner and rule-catalog versions.

[TODO: Describe the implemented parsing strategy, pattern or rule engine, match aggregation, overlapping-match behavior, category mapping, and error handling.]

Relationship and Ordering Algorithm

The relationship stage identifies related artifacts, determines which artifact is the base and which is derived, and records the evidence supporting both decisions. A comparison proceeds only when the relationship and ordering satisfy the eligibility criteria in the Dataset and Data Preparation section.

[TODO: Describe the implemented relationship types, candidate-generation logic, ordering evidence, tie or ambiguity handling, confidence or review rules, and any lineage limitations.]

Tools and Implementation Architecture

The implementation is organized around four functional components:

1. **Extractor:** Reads the relevant records and metadata from `agent_skills_release.db`.
2. **Relationship builder:** Produces candidate ordered base-derived relationships and eligibility evidence.
3. **Static analyzer:** Applies the versioned rule catalog to unique artifact content and creates rule and category profiles.
4. **Comparator and reporter:** Joins profiles to eligible relationships, computes changes and aggregate measures, and writes reproducible analysis artifacts.

[TODO: Name the implemented programming language, database/query engine, packages, command-line entry points, modules or classes, configuration files, test framework, and report-generation tools. Include exact versions or a dependency lock file where applicable.]

End-to-End Pipeline

The intended execution path is:

1. Load the required source tables from `agent_skills_release.db`.
2. Extract analyzable artifact content and traceability metadata.
3. Identify candidate relationships and determine base-derived ordering.
4. Apply eligibility filters and record exclusions.
5. Identify unique content SHAs.
6. Scan each unique SHA with the configured scanner and rule catalog.
7. Join cached profiles back to all applicable artifact occurrences.
8. Compare base and derived profiles to calculate introduced, removed, and retained rules and categories.
9. Produce structured result tables, summary measures, validation inputs, logs, and report-ready figures or tables.

The pipeline records the scanner version, rule-catalog version, configuration, and other run metadata needed to connect every result to the implementation that produced it.

[TODO: Replace this intended flow with the final executable commands and implemented module-to-module data flow when the pipeline is complete.]

Baselines and Test Fixtures

The earlier or source artifact serves as the within-relationship baseline for each derived artifact. This paired baseline supports a direct claim about whether a signal was detected in the derived artifact but not in its identified base; it is not a general-purpose prediction baseline.

The implementation will also use manually constructed base and derived skill pairs as deterministic test fixtures. Fixtures will cover known cases in which rules and categories are introduced, removed, retained, or unchanged. These fixtures test scanner and comparison behavior and are not empirical observations from the GitSkills population.

[TODO: Define any additional baseline rule set, heuristic, comparison method, or ablation required by the final evaluation, and explain what question that baseline answers.]

Design Decisions

Ordered Comparisons Rather Than Isolated Artifacts

The research question concerns behavior newly introduced through modification or reuse. An ordered base-derived pair therefore provides a more direct unit of analysis than an isolated artifact.

Rule-Level and Category-Level Measurement

The method reports both levels because category-only comparisons can hide the introduction of a new rule inside an already-present category, while rule-only results may be too granular to summarize clearly.

One Rule Catalog per Comparison and Run

Scanning both sides with the same version prevents rule-catalog changes from being mistaken for changes in artifact content. When rules change, the catalog version changes and affected analyses must be rerun.

SHA-Based Caching Without Statistical Deduplication

Scanning each unique content SHA once avoids redundant computation. Cached profiles are joined back to all artifact and relationship occurrences so the optimization does not silently redefine the research population.

Census of Eligible Comparisons

The automated pipeline attempts to analyze every eligible comparison. Development subsets are used only for testing and debugging; they do not replace the population-level run.

Evidence Retention and Auditable Filtering

The pipeline retains evidence for relationship detection, ordering, rule matches, eligibility decisions, and exclusions. This supports later manual review and makes the construction of the analyzed population auditable.

Static Analysis and Cautious Interpretation

The project does not execute untrusted artifact content. Detected introductions are reported as static risk signals. They do not, by themselves, prove malicious intent, vulnerability, exploitability, or real-world impact.

Generated Outputs

The pipeline is expected to produce:

- an artifact table with traceability fields;
- a relationship table with ordering evidence and eligibility status;
- versioned scanner profiles;
- an ordered comparison table with introduced, removed, and retained signals;
- exclusion counts and reasons;
- aggregate summary tables;
- a seeded manual-validation sample;
- logs and run metadata; and

- report-ready tables and figures generated from structured results.

[TODO: Replace this expected list with the final filenames, formats, schemas, and repository paths. Identify which outputs are committed, generated on demand, or omitted because of size or sensitive content.]

Reproducibility

A reproducible run must identify:

- the dataset release or snapshot and integrity information;
- the required environment and dependency versions;
- configuration and command-line arguments;
- scanner and rule-catalog versions;
- random seeds used for validation sampling;
- executable commands for each pipeline stage and the complete pipeline;
- output locations and expected artifacts; and
- known platform, resource, or data-availability limitations.

[TODO: Add the final setup instructions and exact clean-environment execution command after the implementation is complete.]

Evaluation and Validation

Evaluation Objectives

[TODO: Explain which parts of the extraction, relationship identification, scanning, and comparison methods require validation.]

Automated Tests

[TODO: Describe unit, integration, and regression tests for important pipeline behavior.]

Constructed Test Fixtures

[TODO: Explain how intentionally created base and derived skills test known rule introductions, removals, and retained behavior.]

Manual Validation Sample

[TODO: Define the sample size, selection method, strata, random seed, and replacement procedure.]

Annotation Protocol

[TODO: Explain what reviewers examine and how cases are labeled.]

Validation Measures

[TODO: Define the validation metrics used, such as true-positive counts, false-positive counts, false-negative counts, precision, recall, or agreement.]

Error Analysis

[TODO: Describe how incorrect, ambiguous, or disputed results are examined and categorized.]

Risk-Signal Interpretation Disclaimer

A detected risk signal indicates that a skill contains an instruction pattern identified as security-relevant by the project's current rule catalog. It does not prove that the skill is malicious, vulnerable, exploitable, or unsafe.

Likewise, an introduced risk signal means that the scanner detected a rule or category in the derived artifact that it did not detect in the corresponding base artifact. This is a comparison of static artifact content, not evidence that the instruction was executed or caused harm.

The available evidence may not establish:

- whether the instruction is executable in the intended environment;
- whether an agent or user ever executed it;
- whether execution requires user approval or additional permissions;
- whether security controls would prevent or restrict the behavior;
- whether a referenced file, script, command, credential, or network resource is harmful;
- whether the behavior creates an exploitable condition; or
- whether the artifact's author intended malicious behavior.

Many legitimate skills require commands, file operations, network access, scripts, or authentication instructions to perform their intended functions. The project will therefore describe scanner findings as **security-relevant capabilities**, **risk signals**, or **patterns requiring review**. Terms such as **malicious**, **vulnerable**, and **exploitable** will be used only when supported by additional evidence beyond a static rule match.

Results

Dataset Processing Results

[TODO: Report artifact counts, unique content SHAs, candidate relationships, eligible comparisons, exclusions, and completed comparisons.]

Security-Pattern Results

[TODO: Report the prevalence of detected rules and categories.]

Introduced Risk Signals

[TODO: Report how frequently derived artifacts introduced rules or categories that were absent from their base artifacts.]

Removed and Retained Signals

[TODO: Report how frequently signals were removed, retained, or unchanged.]

Results by Relationship Type

[TODO: Compare meaningful relationship groups when supported by the data.]

Validation Results

[TODO: Report the results of manual review, including false positives, false negatives, ambiguous cases, and relationship errors.]

Representative Examples

[TODO: Present safe, non-sensitive examples illustrating important or unexpected findings.]

Results Summary

[TODO: Summarize the principal findings without interpreting their broader meaning.]

Discussion

Interpretation of Findings

[TODO: Explain what the results suggest about security-relevant changes in modified or reused GitSkills.]

Comparison With Expectations

[TODO: Compare the findings with the expectations stated earlier in the report.]

Simpler and Competing Explanations

[TODO: Discuss plausible explanations other than insecure or malicious propagation, such as legitimate operational requirements, artifact size, shared templates, or scanner bias.]

Security and Supply-Chain Implications

[TODO: Discuss the implications for skill authors, reviewers, tool developers, repositories, and users.]

Practical Recommendations

[TODO: Present recommendations supported by the findings.]

Unexpected Findings

[TODO: Discuss surprising results and possible explanations.]

Interpretation Boundaries

[TODO: Reinforce what the evidence does and does not establish, including limits involving intent, execution, exploitability, causation, and provenance.]

Threats to Validity

Construct Validity

[TODO: Discuss whether the selected rules and categories accurately represent the security-relevant behaviors the project intends to measure.]

Internal Validity

[TODO: Discuss errors in relationship identification, version ordering, content extraction, rule matching, and profile comparison.]

External Validity

[TODO: Discuss whether findings from the selected dataset generalize to other GitSkills, repositories, agent platforms, or time periods.]

Conclusion Validity

[TODO: Discuss sample size, sparse categories, repeated content, unstable estimates, multiple comparisons, and other factors that could affect the conclusions.]

Reproducibility Threats

[TODO: Discuss dependency versions, dataset availability, rule-catalog changes, nondeterminism, random seeds, and computational requirements.]

Missing and Incomplete Data

[TODO: Explain how unavailable content, incomplete histories, unidentifiable relationships, and excluded records may affect the results.]

Mitigations

[TODO: Summarize the tests, manual validation, versioning, documentation, sensitivity analyses, and reporting practices used to reduce these threats.]

Related Work and References

Mining Software Repositories

[TODO: Summarize relevant research involving repository mining, artifact evolution, reuse, and software change analysis.]

Software Supply-Chain Security

[TODO: Summarize relevant work involving reused artifacts, dependency risk, provenance, and supply-chain propagation.]

Static Security Analysis

[TODO: Summarize relevant work involving rule-based scanning, command detection, configuration analysis, or security-pattern identification.]

AI-Agent Skills and Instructions

[TODO: Summarize relevant research or documentation involving AI-agent skills, prompt-based tools, agent instructions, and AI-native software-engineering artifacts.]

Relationship to This Project

[TODO: Explain how this project differs from or extends the related work.]

References

Conclusion and Future Work

Conclusion

[TODO: Restate the research question, summarize the method, and present the most important findings.]

Contributions

[TODO: Summarize the project's technical, empirical, and methodological contributions.]

Limitations

[TODO: Briefly acknowledge the limitations most important to interpreting the conclusion.]

Future Work

[TODO: Identify reasonable extensions, such as additional rules, improved relationship detection, broader validation, or analysis of other datasets.]

Final Takeaway

[TODO: End with the principal implication for AI-agent skill development, review, or security.]

Artifact Appendix

Repository

[TODO: Provide the repository URL, release tag, commit identifier, and access information.]

Project Structure

[TODO: Describe the important source, test, data, results, figure, documentation, and report directories.]

Environment and Dependencies

[TODO: List the supported operating environment, language version, dependencies, and installation instructions.]

Dataset Availability

[TODO: Explain how to obtain `agent_skills_release.db`, where it should be placed, and why it is not included in the repository.]

Reproduction Commands

[TODO: Provide the exact commands required to run tests, extract data, execute the scanner, perform comparisons, generate results, and build the report.]

Generated Artifacts

[TODO: List the primary result tables, figures, logs, and report outputs and identify which commands generate them.]

Configuration and Versioning

[TODO: Document the scanner version, rule-catalog version, configuration files, random seeds, and other settings associated with the reported results.]

Known Limitations

[TODO: Describe practical limitations affecting installation, execution time, storage, hardware requirements, or supported inputs.]

AI-Use Disclosure

[TODO: Summarize how AI tools were used, how generated work was reviewed, and where the detailed AI-use log can be found.]

Team Contributions

[TODO: Summarize each team member's documented technical, research, review, and project-management contributions.]